

Module 01: Systems — Your Code Is a System

Learning Objectives

1. Understand that a **program** is a system of interconnected parts (variables, functions, loops).
2. Identify the **inputs**, **outputs**, and **internal state** of a simple program.
3. Recognize that a bug in one part of a program can ripple through the whole system.

Introduction

You have been learning about systems since Unit 1 — in your daily life, in science, and in society. Now it is time to *build* one. When you write code, you are creating a system: inputs come in (keyboard, mouse, sensor data), the program processes them (variables change, conditions are checked), and outputs go out (text on screen, sounds, movement in a game). The same ideas that describe ecosystems and solar systems also describe computer programs!

Key Concepts

1. Programs as Systems

Consider a simple game loop:

```
score = 0                # Internal state
while True:
    event = get_input()   # Sensory input
    if event == "catch":
        score += 1        # State update
    draw_screen(score)    # Action/output
```

This tiny program has all the parts of a system: a **state** (score), **input** (the player catching something), **processing** (checking the event and updating the score), and **output** (drawing the

screen). It even has a boundary! The program cannot "see" anything outside of `get_input()` — that function is the program's Markov blanket.

2. Functions as Sub-Systems

When you write `draw_screen(score)`, you are creating a **sub-system**. The `draw_screen` function has its own inputs, its own internal logic, and its own outputs. Complex programs are systems made of smaller systems — just like your body is a system of organs, each of which is a system of cells.

3. Bugs as System Failures

What happens if `get_input()` sometimes returns `None` instead of `"catch"`? Your program crashes — because the `if` statement did not expect `None`. One broken part ripples through the whole system. Finding and fixing bugs is *systems thinking*: tracing the flow of information to find where things went wrong.

Activities



Activity 1: Map Your Code's System

Take any program you have written recently. Draw a diagram showing: (1) What goes IN (inputs), (2) What the program REMEMBERS (state), (3) What processing happens (decisions, loops), and (4) What comes OUT (displays, sounds, files). Label the boundary.



Activity 2: Bug Hunt

Intentionally introduce a bug into a simple program (e.g., change `score += 1` to `score += 10`). Give it to a partner and ask them to find the bug by testing the system. How many tests did they need?

Summary

Every program you write is a system with inputs, state, processing, and outputs. Functions are sub-systems. Bugs are system failures. Thinking about your code as a system helps you design, debug, and improve it.

References

- Sweigart, A. (2015). *Automate the Boring Stuff with Python*. Chapter 1.